Ripa Shah

Week 4 Assignment

Course: INFO 531/ISTA 431: Data Warehousing and Analytics in the Cloud

Module/Week: 4 - Week of September 12, 2022

Topic: ETL Metadata Models; Big Data Analytics

Part 1

A metadata model is important to implement data warehouse; the lack of metadata model can lead to a quality concerns about the data warehouse.
One of the primary goals in the building the data warehouse is to improve the information quality in order to achieve certain business objectives or enhanced decision making capabilities.

Since operational data source and target data warehouse reside in separate places, a continuous flow of data from source to target is critical to maintain data freshness in the data warehouse. Information about the data-journey from source to target needs to be tracked in terms of load timestamps and other load parameters for the sake of data consistency and integrity. This information is captured in a metadata model.

For data warehouse administration, maintenance and usage, a centralized management of metadata is important. To refresh data warehouse, the metadata support is essential to data warehouse maintenance team including ETL developers, database administrators and production support team.
Metadata is utilized in data integration, data transformation, OLAP, data mining and refresh monitoring. It captures information that is useful for business users and back-end support personnel.

Refreshing all tables without first checking for source data changes causes unnecessary loads at the expense of resource usage of database systems. The development of a metadata model that enables some utility stored procedures to identify source data changes means that load jobs can be run only when needed
Metadata is important not only through end-user perspective but also from the standpoint of acquisition, transformation, load, extract and analysis of data warehouse.

In data warehouse there are two kinds of metadata (1) logical metadata and technical (ETL) metadata. The ETL metadata linked to the backend data which transform, extract and load the data. ETL metadata is critical for development , batch cycle refreshes and maintenance of a data warehouse. The development of metadata model that enables some stored procedures to identify source data changes that means that

load jobs can be run only needed. The metadata model provides the capability to setup subsequent cycle run behavior followed by the one-time full refresh.

In the metadata model architecture, the load jobs are skipped when source data has not changed. Metadata provides information to decide whether to run full or delta load stored procedures. It also has the capability to force a full load if needed. The model also controls collect statistics jobs running them after a certain interval or on an on-demand basis, which helps minimize resource utilization. The metadata model has several audit tables to archive critical metadata for three months to two years or more. Metadata increases the state of adoption and use of data warehouse by knowledge workers and managers. Metadata also holds errors and message logs for troubleshooting purposes. Also, metadata take care of data quality that users could use it to achieve a specific task. A wrapper stored procedure is a procedure that executes several utility procedures to make load type decisions and, based on that, it runs full or delta stored procedure or skips the load. If source data changes are detected, the wrapper stored procedures call full or delta (aka, incremental) stored procedures based on load condition of each table load. It captures the begin timestamps and end timestamps to track the refresh cycle information.

Sl No.	Entity Name	Entity Description
1	Cycle Log (cyc_log)	Holds subject area refresh begin and end date and time
2	Cycle Log History (cyc_log_hist)	Archives data from table 'cycle log' (#1) for last two years on a rolling basis
3	Job Description (job_desc)	Stores batch cycles job description
4	Load Check Log History (load_chk_log_hist)	Archives load parameters from table 'load check option' (#6) for last two years on a rolling basis
5	Load Configuration (load_config)	Holds threshold values to trim the archive tables (#2, 4, 8, 11) on a rolling basis.
6	Load Check Log (load_chk_log)	Holds load parameters for each table under batch cycles
7	Source Load Log (src_load_log)	Holds last source data change timestamp for each source table used to load target tables.
8	Message Log History (msg_log_hist)	Archives data from table 'message log' (#9) for last two years on a rolling basis
9	Message Log (msg_log)	Loads error messages/ information for each SQL in a procedure used to load a table. Used to troubleshoot job failures.
10	Source Table File Information (src_tbl_file_info)	Holds source data file information. Used to set up load parameters for target tables in a batch cycle in DW
11	Load Metric History (load_mtric_hist)	Archives data from table 'load metric' (#12) for last two years on a rolling basis
12	Load Metric (load_mtric)	Holds detailed load metrics for each target table load
13	Load Check Option (load_chk_opt)	Holds load parameters to determine load behavior of subsequent cycle refreshes for each table load

Table 1. Metadata model entity description.

Wrapper stored procedure algorithm

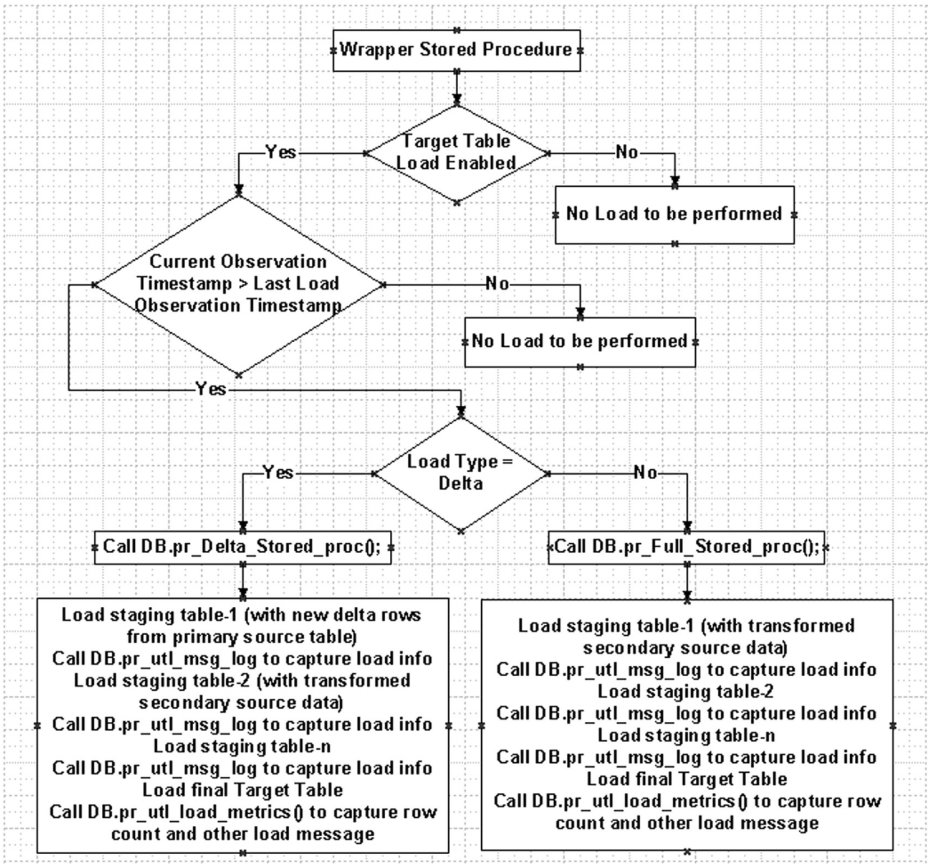

Figure 3. High level software architecture for full and delta load.

Normally, if the target tables requires to insert more than one million rows then the full refresh is needed. For some reason, the target table seems empty then delta stored procedure will call the full stored procedure. Also, delta refreshes performed when full refreshes performed badly and can't be practically optimized further. Once target table load is performed, the full and delta procedure has been implemented then the next step would be to call the utility stored procedure. In order to update metadata table to capture the observation timestamps for source table against the target table, the utility procedure has to be implemented. This whole process is utilized when the data warehouse is not dependent on any ETL tools.

Of course, the first load of any ETL analytical table would be the full refresh. After, according to the requirement, they perform the incremental load. The wrapper procedure first calls a utility procedure to get the last source table load timestamp from the metadata table.

```
SELECT CAST(MAX(CAST(a.asof_src_dt AS CHAR(10) ) ||' '|| CAST(a.asof_src_tm AS CHAR(8) ) ) AS TIMESTAMP(0) )
INTO :src_cur_ts
FROM DWmgr.v_dwm_subj_ara_tbl_fl a, DWmgr_Xfrm_MET.v_src_load_log b
WHERE a.subj_ara_nm = b.src_subj_area_nm
AND a.tbl_nm = b.src_tbl_nm
AND a.row_cnt > 0
AND b.trgt_subj_area_nm = :subj_ara
AND b.trgt_tbl_nm = :trgt_tbl
AND b.trgt_tbl_line_nbr = :min_tbl_line
AND ((a.asof_src_dt = :last_vw_swap_dt AND a.asof_src_tm < :last_vw_swap_tm ) OR (a.asof_src_dt < :last_vw_swap_dt ));
```

Figure 5. Code block that detects source data change.

	TargetSubjArea	TargetTable	SourceSubjArea	SourceTable	LastObservationTimestamp
1	Asset_DRV	asset	Asset_CRS	asset_main_nbr	4/11/2011 11:00:00
2	Asset_DRV	asset_depr_fcst	Asset_CRS	depr_accr_frcst	4/11/2011 11:33:23
3	Capital_DRV	reltn_acct_co	Organization_DRV	acct_co	4/11/2011 10:33:57
4	Capital_DRV	reltn_equip_usr_sts	Project_DRV	equip	4/11/2011 11:44:41
5	Project_DRV	equip_prcss	Characteristics_DRV	equip_mtrl_char	4/11/2011 07:07:36
6	Project_DRV	invst_mgmt_prog_wbs	Project_CRS	invst_prog_obj_asgn	4/11/2011 03:00:00

Table 2. Last observation timestamp for target table refresh.

The msg_log stores the troubleshooting information.

	subj_area_nm	trgt_tbl_nm	forc_load_ind	load_type_cd	load_enable_ind	load_thrhld_nbr	copy_back_ind
1	Asset_CRS_S	depr_accr_frcst	N	Full	Y	0	Y
2	Project_DRV	equip_prcss	N	Full	Y	0	Y
3	Capital_DRV	reltn_equip_usr_sts	N	Full	Y	0	Y
4	Capital_DRV	reltn_acct_co	N	Full	Y	0	Y
5	ePRT_DRV	fact_supl_invc_exp_line	N	Full	Y	0	Y
6	Project_DRV	invst_mgmt_prog_wbs_elem	N	Full	Y	0	Y
7	Asset_DRV	asset_depr_fcst_s	N	Full	Y	10	Y
8	Finance_SHR_DRV	dim_wbs_shr	N	Full	Y	0	Y
9	Capital_DRV	reltn_bldg_plnt	N	Full	Y	0	N

Table 3. Load parameters used to determine load condition.

	subj_area_nm	trgt_tbl_nm	msg_ts	msg_nbr	msg_src	msg_1_txt	msg_2_txt
1	Capital_DRV	dim_asset_chg_log	4/11/2011 12:13:04	51	pr_Ddim_asset_chg_log	SOURCE TABLE: Asset.v_asset_chg_log_line_DRV - CDPOS	INSERTED: appl_Capital_DRV_01.gt_dim_asset_chg_log - ROW COUNT = 149
2	Capital_DRV	dim_asset_chg_log	4/11/2011 12:13:23	54	pr_Ddim_asset_chg_log	SOURCE TABLE: Asset.v_asset_chg_log_hdr_DRV - CDHDR	INSERTED: appl_Capital_DRV_01.gt_dim_asset_chg_log1 - ROW COUNT = 51
3	Capital_DRV	dim_asset_chg_log	4/11/2011 12:13:42	55	pr_Ddim_asset_chg_log	SOURCE TABLE: appl_Capital_DRV_01.gt_dim_asset_chg_log	INSERTED: Capital_DRV_STG.dim_asset_chg_log; ROW COUNT = 148
4	Capital_DRV	dim_asset_chg_log	4/11/2011 12:14:21	56	pr_Ddim_asset_chg_log	SOURCE TABLE: Capital_DRV_STG.dim_asset_chg_log	INSERTED: Capital_DRV_STG.dim_asset_chg_log1; ROW COUNT = 148
5	Capital_DRV	dim_asset_chg_log	4/11/2011 12:14:27	57	pr_Ddim_asset_chg_log	SOURCE TABLE: Capital_DRV_STG.dim_asset_chg_log1	DELETED: Capital_DRV_MET.v_dim_asset_chg_log; ROW COUNT = 0
6	Capital_DRV	dim_asset_chg_log	4/11/2011 12:14:32	58	pr_Ddim_asset_chg_log	SOURCE TABLE: Capital_DRV_STG.dim_asset_chg_log	INSERTED: Capital_DRV_MET.v_dim_asset_chg_log; ROW COUNT = 148

Table 4. Message log.

The utility stored procedure ‘pr utl Load Metrics()’ is called from ’Full’ and ’Delta’ data load stored procedures to insert entries into the ‘load mtric’ table during each target table load. The table ‘load config’ (subj area nm, trgt tbl

nm, obj type, config thrhld nbr, config txt) provides space to hold any staging data or input in order to prepare for a full or delta load. This is also used to trim the history tables on a rolling basis. The config thrhld nbr field holds the threshold value for history tables. It helps to improve load performance in terms of response time and database system's CPU and IO resource consumption.